

git --cheatsheet-notes.tree

Personal reference notes & command hierarchy map // Created for quick lookup

🌳 1. Setup & Configuration (`git config`, `git init`)

- | `git config --global user.name "Name"` — Set global username for commits.
- | `git config --global user.email "email@example.com"` — Set global user email.
- | `git config --list` — View all active configuration settings.
- | `git init` — Initialize a new, empty Git repository in current folder.
Creates hidden `.git` folder containing all project version history.
- | `git clone <repo-url>` — Copy an existing remote repository to local machine.

🌳 2. Basic Snapshotting / Daily Workflow

- | `git status` — Check working tree status (untracked, modified, staged files).
- | `git add <file>` / `git add .` — Stage changes for next commit.
`git add .` stages all modified/new files in project root.
- | `git commit -m "message"` — Record staged snapshot permanently in history.
- | `git commit -am "message"` — Stage modified files & commit in a single shortcut.
- | `git diff` — Show unstaged differences between working dir and staging area.
Use `git diff --staged` to view staged changes before committing.

🌳 3. Branching & Merging

- | `git branch` — List local branches. Active branch has `*`.
- | `git branch <branch-name>` — Create a new branch without switching to it.
- | `git checkout <branch>` / `git switch <branch>` — Switch active branch.
- | `git checkout -b <branch>` / `git switch -c <branch>` — Create AND switch to new branch immediately.
- | `git merge <branch>` — Combine specified branch into current active branch.
- | `git rebase <base-branch>` — Re-apply commits on top of another base tip for linear history.
- | `git branch -d <branch>` — Safely delete a merged branch (`-D` forces deletion).

🌳 4. Remote Repositories (`git remote`, `push`, `pull`)

- | `git remote add origin <url>` — Connect local repo to remote URL named "origin".
- | `git remote -v` — Show registered remote URLs (fetch & push).
- | `git fetch` — Download changes from remote without merging into current working code.
- | `git pull` — Fetch and automatically merge remote updates into local branch (`fetch + merge`).
- | `git push -u origin <branch>` — Push local commits to remote and set upstream tracking branch.

5. Inspection, History & Log

- | `git log` — View commit history.
- | `git log --oneline --graph --all` — Compact visual tree layout of all branch histories.
- | `git show <commit-hash>` — Inspect details & changes of a specific commit.
- | `git blame <file>` — Line-by-line author attribution & commit ID lookup.

6. Undoing Changes & Fixing Mistakes

- | `git restore <file>` — Discard local unstaged changes in working file.
- | `git restore --staged <file>` — Unstage file while preserving changes in local file.
- | `git commit --amend -m "new msg"` — Modify last commit message or add staged changes to it.
- | `git reset --soft HEAD~1` — Undo last commit, keep changes staged.
- | `git reset --hard HEAD~1` — Permanently erase last commit and all uncommitted changes!
- | `git revert <commit-hash>` — Create a new commit that safely undoes effects of target commit.

7. Advanced Utilities (Stash, Reflog, Cherry-Pick, Tags)

- | `git stash` / `git stash pop` — Temporarily shelf dirty working changes / restore stashed changes.
- | `git reflog` — Safety log of all HEAD movements (helps recover lost commits).
- | `git cherry-pick <commit-hash>` — Apply specific commit from another branch into current branch.
- | `git tag -a v1.0 -m "Release"` — Create annotated tag for milestone/release versions.